
Submitting jobs to the queue system

Version 126.1 authored by Anna Maria Asplund [Anna Maria Asplund](#) on 2024/08/23 15:25

- Introduction
 - Batch job
 - Available nodes
- User creation and modification
- Logging into the system for the first time
- Username and password
- How to use the system
 - 1. Connect to a login node
 - From Citrix-NGC
 - From Self-manged cloud
 - How to submit a batch job/script
 - Example on how to create and submit a shell script with qsub
 - Submitting many jobs simultaneously
 - How to monitor, manage and cancel submitted jobs
 - Estimating job resource requirements
 - Check resources used using qstat -f
 - Check resources used using checkjob
 - Waiting for a specific job to finish before starting the next one
 - Submitting jobs to specific nodes
 - Searching for free resources
 - Writing a PBS batch scripts
 - How to write a PBS batch script
 - PBS directives
- Conclusion
 - Relevant links
- Error handling
- Support

Introduction

An HPC is a multi-user environment. To ensure effective usage of the available resources, a job-scheduler (aka resource manager) is used to manage the workload. The job-scheduler assigns resources to users according to the system's current and expected load. This means that users do not run their job directly, but instead "ask" the system to run their job, usually by the means of a batch job.

Batch job

A batch job is a tool/program or set of tools/programs processed in batch mode. This means that a sequence of commands, to be executed by the operating system, is listed in a file, often referred to as a batch script, command file, job script, or shell script. The batch script is submitted for execution, or sent to the job queue, as a single unit. The job-scheduler parses the script and tries to optimize the usage of the available resources, scheduling the execution of all incoming batch jobs at different times, and on different nodes/cores in the HPC cluster. At NGC we use TORQUE for submitting and monitoring jobs (resource manager), and the Moab Workload Manager for dispatching jobs (scheduler).

The opposite of a batch job is interactive processing, in which a user enters individual commands to be processed immediately. [Here](#) (link only accessible from NGC-Wiki) is a guide on how to start an interactive session. An interactive session is suitable for developing and testing a batch script.

Available nodes

To see available nodes on your cloud you can run the command:

```
# List info about available nodes
pbsnodes
```

User creation and modification

To access this service you need to become a diagnostic user (clinical services) or research user (NGC-managed cloud or Self-managed cloud) of the NGC services.

[Here](#)  (link only accessible from the NGC Wiki) you can read about how to become a user of NGC services.

As a user on the NGC-HPC or an NGC-hosted self-managed cloud you have access to submitting jobs to the queue system by default.

Logging into the system for the first time

No special requirements.

Username and password

There is no password restriction on this solution as long as you are logged in to Citrix-NGC (<https://secureremote.ngc.dk>) and NGC-HPC or VMWare Horizon and a Self-manged cloud, you will have access to submitting batch jobs (see details below).

How to use the system

1. Connect to a login node

From Citrix-NGC

This solution is for diagnostic (clinical services) users and research users on **NGC-managed cloud**.

First you need to log in to Citrix-NGC (<https://secureremote.ngc.dk>). [Here](#) (link only accessible from the NGC Wiki) is a guide on how to log in to Citrix-NGC.

Then you need to access NGC-HPC.

Alternative 1) Open the MobaXterm app and log in to NGC-HPC through one of the login nodes.

Alternative 2) Open NGC Desktop, then open MobaXterm from the desktop icon and log in to NGC-HPC through one of the login nodes.

See this guide for more details on how to complete the above steps: [NGC Desktop and NGC-HPC](#) (link only accessible from the NGC Wiki)

From Self-manged cloud

This solution is for research users with access to **Self-managed cloud**.

Open VMware Horizon Client (<https://spc.ngc.dk> is the connection server).

Enter your NGC username and password and click **Login**.

You will receive a notification on your phone from Entrust Identity app. Click on the app

notification, login with your chosen PIN (or face recognition) and choose **Confirm**.

You are now logged into VMware Horizon client and can see an overview of the services you have access to.

Double click on COS-CLDXXX to login to a CentOS desktop.

Open the Terminal (double click the Terminal icon on the desktop) and ssh into a login node. Enter your password.

```
# ssh into a Login node
ssh -X <username>@cld<cloud_id>-0001
# example with user jondoe on cloud 099
ssh -X jondoe@cld099-0001
```

The **-X** option makes it possible to open graphical user interface tools like e.g. Firefox from your session.

How to submit a batch job/script

You can submit batch jobs via the command `qsub`. The general command for submitting a job/script into the queue system is as follows:

```
qsub -W group_list=<group_name> -A <group_name> -l nodes=<no_nodes>:ppn=
<no_CPUs>,mem=<memory>,walltime=<hrs:min:sec> script.sh
```

The `qsub` options specified in the above command are the following:

1) How long time you expect the job to run: **-l walltime=<time>**. `<time>` is specified in seconds (i.e. `walltime=3600` corresponds to an hour), but can also be specified in the format `hours:minutes:seconds` (i.e. `walltime=01:00:00` corresponds to an hour).

2) How much memory the job requires: **-l mem=<Xgb>**

3) How many nodes and CPUs is needed: **-l nodes=<#nodes>:ppn=<#CPUs per node>**

The number of CPUs (`ppn=processors per node`) can range from 1 to 40.

4) The group name of your department/team/project: **-W group_list=<group_name> -A <group_name>**

Some `qsub` examples:

```
# submitting a 1-CPU job
qsub -W group_list=<group_name> -A <group_name> -l nodes=1:ppn=1 <your script>

# run a job (e.g. script.sh) with 20 CPUs, 100GB memory for 1 hour for group_VIP
qsub -W group_list=group_VIP -A group_VIP -l nodes=1:ppn=20,mem=100gb,walltime=3600
script.sh

# run a small 10 min, 1 node, 1 CPU job for group_VIP
qsub -W group_list=group_VIP -A group_VIP -l nodes=1:ppn=1,walltime=00:10:00
script.sh

# for more options on using qsub
man qsub
```

Note

Do not request all the memory of a node as the node OS uses some of the memory. For a thin (or GPU) node, allocating maximum 185 GB of memory to the actual job would be OK.

Example on how to create and submit a shell script with qsub

Create a shell script using Vim.

```
# open (or create and open if not already exiting) the script samtools_script.sh in
the text editor program Vim
vim samtools_script.sh
```

Press **i** for insert.

Then add your commands to the script.

Example shell script running samtools for converting a binary bam file into a human readable sam file:

```
#!/bin/bash

# Load the Samtools module
module load samtools/1.14
```

```
# Change to the directory containing your input files
cd /path/to/input/files/

outdir="/path/to/output/files/"

# Run samtool view command and redirect the output to output.sam
samtools view -h input.bam > ${outdir}/output.sam
# -h includes the header in the output

# Run additional commands on the output file if desired
```

Press **Esc** to exit the insert mode in Vim and press **:x** to exit Vim and save your changes.

From one of the login nodes on NGC-HPC submit your shell script using qsub.

```
# submit shell script with qsub
qsub -W group_list=<group_name> -A <group_name> -N bam_to_sam -l
nodes=1:ppn=1,mem=20gb,walltime=1:00:00 -d $(pwd) samtools_script.sh
# e.g.
qsub -W group_list=ngc-bio -A ngc-bio -N bam_to_sam -l
nodes=1:ppn=1,mem=20gb,walltime=1:00:00 -d $(pwd) samtools_script.sh
```

The option **-N bam_to_sam** defines the name of the job to be "bam_to_sam", which will also be the name of the standard out and standard error file which are also created with the qsub command. In these files you can find useful information in regards to debugging your script as they will contain error messages and output text from your shell commands.

The option **-d \$(pwd)** defines the working directory path to be the current directory. This is where the standard out and standard error file from the executed batch job/script will be created. This is also where relative paths in the script are relative from. If this is not set, the working directory path will by default be your home directory (/ngc/people/<NGC-username>).

Submitting many jobs simultaneously

Not to overload the queue system, and preventing your jobs from being blocked, it is a good idea to incorporate a pause between job submissions. In a shell script you can use the command "sleep <seconds>" to make the execution of the next command delay for the specified number of seconds. Here is an example of a shell script with an incorporated pause:

Job submission script with sleep

```
#!/bin/bash

# Submit job1
qsub -W group_list=ngc-bio -A ngc-bio -l nodes=1:ppn=5,mem=10gb,walltime=1:00:00
script1.sh

# Wait for 30 seconds
sleep 30

# Submit job2
qsub -W group_list=ngc-bio -A ngc-bio -l nodes=1:ppn=5,mem=10gb,walltime=1:00:00
script2.sh

# Wait for 30 seconds
sleep 30

# Submit job3
qsub -W group_list=ngc-bio -A ngc-bio -l nodes=1:ppn=5,mem=10gb,walltime=1:00:00
script3.sh

# Wait for 30 seconds
sleep 30

# Submit job4
qsub -W group_list=ngc-bio -A ngc-bio -l nodes=1:ppn=5,mem=10gb,walltime=1:00:00
script4.sh

# Wait for 30 seconds
sleep 30

# Submit job5
qsub -W group_list=ngc-bio -A ngc-bio -l nodes=1:ppn=5,mem=10gb,walltime=1:00:00
script5.sh
```

How to monitor, manage and cancel submitted jobs

In the following, you will find useful commands in regards to monitor, manage and cancel submitted jobs.

```
# get the status of your submitted jobs (Q: in queue but not yet running; R:
running, C: complete)
qstat
```


Submitting jobs to the queue system

```
# get extended status of your submitted jobs like requested resources and elapsed time
qstat -a

# see the nodes allocated to each run
qstat -n

# get the status of a specific job
qstat <jobid>
# note: jobid is the value in the first column of output from qstat

# more info about a specific job
qstat -f <jobid>
# with this option you can check used time and memory of a completed job (see resources_used.walltime and resources_used.mem)

# to get more options, check the manual
man qstat

# view the qsub queue, including qstat all users
showq

# view running jobs (+ give additional info, including an estimate on how efficiently they are using the CPUs)
showq -r

# this will give you more information about your eligible jobs
showq -i

# for more options with the showq command:
showq -h

# if you want to check the status of a particular jobid
checkjob <jobid>

# add the -v flag to increase the verbosity
checkjob -v <jobid>

# check when a job will run
showstart <jobid>

# check resource usage of completed job (privileged command)
tracejob -v <jobid>

# to cancel a job, first find the job-id using qstat, then type:
qdel <jobid>
```

for more options with the qdel command:

```
man qdel
```

to cancel a job using mjobctl, first find the job-id using qstat, then type:

```
mjobctl -c <jobid>
```

The <jobid> can be the whole string below the **Job ID** column, e.g. 12345678.i-07-moab1.et. However if you get an error message like "ERROR: invalid job specified: 12345678.i-07-moab1.et", try with only the eight digit number as <jobid>.

Estimating job resource requirements

The job-scheduler reserves the requested resources for your job, and run it when there are enough resources (avoiding conflicts with other users' jobs). It is, therefore, important that the resources requested correspond to what really is needed by your job.

First time you run your script you may not have a clear picture of what kind of resource requirements it has. To get a rough estimate, you can submit a job to a full node, with large walltime (e.g. 48 hours).

Regular compute node, aka. thinnode

test run, using a full thin node, to estimate resources needed

```
qsub -W group_list=<group_name> -A <group_name> -l
```

```
nodes=1:ppn=40:thinnode,walltime=48:00:00 <script>
```

with specified memory resources

```
qsub -W group_list=<group_name> -A <group_name> -l
```

```
nodes=1:ppn=40,mem=185gb,walltime=48:00:00 <script>
```

For very compute heavy jobs you can also try running at test run on a fat node. Fat nodes are very limited in numbers and in high demand. Therefore fat nodes should only be used when the memory usage is more than what is available on a thin node. Only when the memory usage is above (or very close to the maximum of) the resources available on a thin node, a fat node will make your job run faster and/or not crash.

Fat node

```
# test run, using a full fat node, to estimate resources needed
qsub -W group_list=<group_NAME> -A <group_NAME> -l
nodes=1:ppn=40:fatnode,walltime=48:00:00 <script>
# with specified memory resources
qsub -W group_list=<group_name> -A <group_name> -l
nodes=1:ppn=40,mem=1500gb,walltime=48:00:00 <script>
```

Interactive session

The resources used can also be checked after having exited an interactive session. This can guide you on the resource demand for setting up future interactive sessions when e.g. running Rstudio or Jupyter Notebook.

Check resources used using qstat -f

When your job has completed (and up to 12 hours after) you can check the actual resource usage. Identify the jobid by running "qstat -a", and then see output from "qstat -f <jobid>".

```
$ qstat -a
```

```
i-07-moab1.eth.clb:
```

Job ID	Req'd Memory	Elap Time	S	Username	Queue	Jobname	SessID	NDS	TSK
40212422.i-07-moab1.et	20gb	01:00:00	C	<user>	batch	cram_to_sam	6864	1	1
40212526.i-07-moab1.et	20gb	01:00:00	C	<user>	batch	bam_to_sam	27509	1	1

```
$ qstat -f 40212526
```

```
Job Id: 40212526.i-07-moab1.eth.clb
Job_Name = bam_to_sam
Job_Owner = <user>@i-04-l0001.eth.clb
resources_used.cput = 00:00:15
resources_used.vmem = 197704kb
resources_used.walltime = 00:02:25
resources_used.mem = 11596kb
resources_used.energy_used = 0
job_state = C
queue = batch
server = i-07-moab1.eth.clb
Account_Name = ngc-bio
...
```

The \$ symbol is to indicate command lines in the terminal. Lines without the \$ symbol is standard out to the terminal.

Check resources used using checkjob

You can also check resources used for a job with "checkjob -v <jobid>", -v option stands for verbose and ads information about memory usage.

```
checkjob -v <jobid>
# e.g.
checkjob -v 12345678
```

Checkjob output example:

```
$ checkjob -v 40213371
job 40213371 (RM job '40213371.i-07-moab1.eth.clb')

AName: cram_to_bam
State: Completed
Completion Code: 1   Time: Tue Mar 28 14:35:36
Creds:  user:<user>  group:ngc-bio  account:ngc-bio  class:batch
WallTime:   2:57:46 of 2:00:00:00
SubmitTime: Tue Mar 28 11:37:35
  (Time Queued  Total: 00:00:15  Eligible: 00:00:15)

TemplateSets:  DEFAULT
NodeMatchPolicy: EXACTNODE
Total Requested Tasks: 20
Total Requested Nodes: 1

Req[0]  TaskCount: 20  Partition: pbs
Dedicated Resources Per Task: PROCS: 1  MEM: 4608M
GMetric[energy_used]  Current: 0.00  Min: 0.00  Max: 0.00  Avg: 0.00  Total: 0.00
Average Utilized Memory: 45.16 MB
Average Utilized Procs: 5.66
...
```

To calculate what you should use for the **-l mem=** parameter, you should multiply the number of tasks (CPUs) with "Average Utilized Memory".

In this case it would be $45.16\text{mb}/\text{cpu} * 20\text{cpu} = 900\text{mb}$.

When assigning memory to your future submissions of the same batch job/script, it is recommended to add some extra memory resources to your job (e.g. 10-20% extra) as the resource usage can fluctuate somewhat from run to run.

Waiting for a specific job to finish before starting the next one

It is possible to specify that a job should only run after another job has completed successfully, please see the **-W** flags in "man qsub".

To run your script after job 12345678 has completed successfully:

```
qsub -W depend=afterok:<jobid> <script>  
# e.g.  
qsub -W depend=afterok:12345678 script_2.sh
```

Be sure that the exit status of job 12345678 is meaningful; if it exits with status 0, your second job will run. If it exits with any other status, your second job will be cancelled. It is also possible to run a job if another job fails (`afternotok`) or after another job completes, regardless of status (`afterany`). Be aware that the keyword "after" (as in "-W depend=after:12345678") means run after job 12345678 has **started**.

Submitting jobs to specific nodes

Thin and fat nodes differ in available memory. Thin nodes have 192 GB of RAM (less available for allocation to the actual job, 185gb OK) while fat nodes have 1.5 TB of RAM. Both node types have 40 CPU cores. For self-managed cloud services the type and number of nodes available depends on the specific solution.

```
# submitting a job to two entire fat nodes  
qsub -W group_list=<group_name> -A <group_name> -l nodes=2:ppn=40:fatnode <your  
script>
```

```
# submitting a job to two entire fatnodes, 80 CPUs and ~3.0 TB RAM in total  
qsub -W group_list=<group_name> -A <group_name> -l  
nodes=2:ppn=40:fatnode,mem=3000gb <your script>
```

```
# submitting a job to two entire thin nodes (i.e., regular nodes)  
qsub -W group_list=<group_NAME> -A <group_NAME> -l nodes=2:ppn=40:thinnode <your  
script>
```

```
# submitting a job to two entire thinnodes, 80 CPUs and ~370 GB RAM in total  
qsub -W group_list=<group_name> -A <group_name> -l nodes=2:ppn=40:fatnode,mem=370gb  
<your script>
```

You can also specify the node that you want to use for your job:

```
qsub -W group_list=<group_NAME> -A <group_NAME> -l nodes=<node-id>:ppn=40 <your  
script>
```

Note

On NGC-HPC there are fewer available fat nodes why you may need to wait for a longer time to get access to a fat node than a thin node.

Searching for free resources

Show what resources are available for immediate use.

```
# fat nodes
showbf -f fatnode

# thin nodes
showbf -f thinnode
```

Writing a PBS batch scripts

It is possible to set the qsub options in your batch job script. The qsub options are specified in a simple text file, following a specific syntax. Once you have this text file (the job script), you submit it to the queue by typing in the terminal the command "qsub <pbs_script>".

How to write a PBS batch script

A batch script contains a series of commands to be executed. For instance, a typical PBS (Portable Batch System) .pbs script can look something like this:

```
#!/bin/bash

#PBS -W group_list=ngc-bio
#PBS -A ngc-bio
#PBS -N job_name
#PBS -l nodes=1:ppn=8
#PBS -l mem=100gb
#PBS -l walltime=3:00:00
#PBS -e $PBS_JOBID.err
#PBS -o $PBS_JOBID.Log

cd $PBS_O_WORKDIR
module load anaconda3/2022.10
```

```
python /ngc/projects/group_name/scripts/python_script.py
```

There are four sections:

- 1)** The first line (`#!/bin/bash`) tells the system that this is a shell script (if in doubt, type "which bash" into the terminal).
- 2)** In the second block (lines starting with `#PBS`) you specify the resources needed for the job (instead of in the `qsub` command when submitting the job), as well as specify names of standard output and standard error files from the program. The `$PBS_JOBID` will ensure unique filenames for each job.
- 3)** The third section creates the proper environment to run the job: it loads the module(s) and changes the working directory to the directory from which the job was submitted (you can also specify the full path).
- 4)** The fourth part contains the script that will be executed. This can be a python script as in the example, but it could as well be a whole bash script containing a series of commands, or another program.

Note

There is a maximum size limit of 64KB for scripts submitted to the queue system. Scripts submitted to the queuing system should mainly consist of parameters for the queuing system and executions of the "real" job, which could be e.g. a python script (as above) or a shell script.

PBS directives

PBS directive	What it does
#PBS -q <queue name>	Specifies that the job will be submitted to the <queue name> queue
#PBS -l nodes=<n>:ppn=<p>	Requests <n> node(s), <p> CPU(s) per node
#PBS -l mem=<m>	Request the total amount of memory (m) needed across all nodes. Default units are bytes; can also be expressed in megabytes (mem=4000MB) or gigabytes (mem=4GB).
#PBS -l wall-time=hh:mm:ss	Sets max wall time for the job to hh:mm:ss
#PBS -l nodes=<n>:ppn=<p>:gpu	Requests a GPU node
#PBS -N <job name>	Sets the job name, which appears in status listings and is used as the prefix in the job's output and error log files. The job name must not contain spaces.
#PBS -o <file name>.out	Sends standard output to <file name>.out
#PBS -e <file name>.err	Sends standard error to <file name>.err
#PBS -j oe	By default, PBS returns two log files, one for the standard output stream (stdout), the other for the standard error stream (stderr). This option joins both into a single log file.
#PBS -S /bin/\$shell	Sets the shell to be used in executing your script. If left out, it defaults to your normal login shell. Typical values for the \$shell argument are /bin/bash, /bin/tcsh, /bin/csh or /bin/sh.
#PBS -V	Export all environment variables in the qsub command environment to the batch job environment.

<code>#PBS -a [YYYY] [MM][DD]hhmm</code>	Delay executing batch job until a particular date and time. Hours and minutes are required; year, month and day are optional. No spaces are allowed in the numeric string. (e.g. <code>#PBS -a 1700</code> will run the job after 5pm.)
<code>#PBS -W de- pend=afterany: <jobid></code>	This job may be scheduled for execution only after job <code>jobid</code> has terminated.
<code>#PBS -A <project ac- count></code>	Identifies project job resources will be charged against.

Conclusion

You should now be able to submit a shell or PBS script as a batch job to NGC-HPC or your NGC-hosted self-managed cloud using `qsub`.

Relevant links

For development of batch script it is recommended to use an interactive session, see this guide:

[Interactive session](#) (link only accessible from the NGC Wiki)

Error handling

Nothing relevant.

Support

You can get help with issues related to the NGC services by writing an e-mail to support@ngc.dk. You can significantly speed up the support process by providing complete and detailed information for faster troubleshooting and error correction.

In the email to support@ngc.dk please include the following information (as detailed as you can in relation to your knowledge):

- An informative e-mail subject line.
- Your name, NGC username, and the [project \(user group\) you belong to](#).
- When did the problem occur, how often does it occur and duration of issue.
- Which cluster? Which node? Which job?
- What happened? What exactly were you doing/trying to do? Include information on which commands you were using and/or which tools/modules (incl. versions).